

BLUEPRINT APLIKACJI DESKTOPOWEJ

AI Code Assistant Desktop App

Kompletny design, stack technologiczny (Tauri v2)
i szczegolowy roadmap dla aplikacji typu T3 Code / Codex

WERSJA

2.0 - Desktop

DATA

16.05.2026

TYP DOKUMENTU

Design + Stack + Roadmap

Spis treści

1. Wprowadzenie i Cel Aplikacji

2. Design Aplikacji - Co Powinna Zawierac

- 2.1. Architektura Okna i Layout
- 2.2. Panel Projektu (Sidebar)
- 2.3. Panel Watkow (Thread List)
- 2.4. Panel Chatu (Główny)
- 2.5. Diff Viewer i Code Review
- 2.6. Zintegrowany Terminal
- 2.7. System Trybow Pracy
- 2.8. System Skili i Quick Actions
- 2.9. Integracja Git i PR
- 2.10. Ustawienia i Konfiguracja

3. Pełny Stack Technologiczny

- 3.1. Desktop Shell - Tauri v2
- 3.2. Frontend - React 19
- 3.3. Backend - Rust
- 3.4. Baza Danych - SQLite
- 3.5. AI / LLM Layer
- 3.6. Code Editor - Monaco
- 3.7. Terminal - xterm.js
- 3.8. Infrastruktura i DevOps

4. Architektura Systemu

5. Roadmap Projektowy

- 5.1. Faza 0: Setup (Sprint 1)
- 5.2. Faza 1: Core UI Layout (Sprint 2-3)
- 5.3. Faza 2: Chat AI (Sprint 4-5)
- 5.4. Faza 3: Projekty i Git (Sprint 6-7)
- 5.5. Faza 4: Agenci i Worktree (Sprint 8-10)
- 5.6. Faza 5: Diff, Terminal i Skille (Sprint 11-13)
- 5.7. Faza 6: Hardening i Release (Sprint 14-16)

6. Szczegolowy Opis Sprintow

7. Wymagania Systemowe

1. Wprowadzenie i Cel Aplikacji

Cel dokumentu

Niniejszy dokument stanowi kompletny blueprint do stworzenia **desktopowej aplikacji AI Code Assistant** - narzędzia, które będzie twoim prywatnym, lokalnym centrum dowodzenia do pracy z agentami AI kodującymi. Aplikacja jest inspirowana **T3 Code** (Theo) i **OpenAI Codex App**, ale budowana od zera jako twoje własne rozwiązanie.

Dla kogo jest ta aplikacja

Dla ciebie - solo developera, który chce mieć **własne, prywatne narzędzie** do pracy z AI kodującym. Nie zależy ci na SaaS, subskrypcjach czy cloudzie. Chcesz aplikację desktopową, która:

- Działa **lokalnie** na twoim komputerze (z możliwością cloud API gdy chcesz)
- Łączy się z **twoimi własnymi kluczami API** (BYOK - Bring Your Own Key)
- Pozwala pracować nad **wieloma projektami Git** równocześnie
- Izoluje zmiany AI w **Git worktree** (nie brudzi twojego głównego workspace)
- Pozwala uruchamiać **wielu agentów równolegle** na różnych zadaniach
- Ma **zintegrowany diff viewer** do review zmian
- Ma **zintegrowany terminal** do komend

Philosophia aplikacji

"**Twoje AI, twoje klucze, twój kod, twoja maszyna.**" Aplikacja nie jest kolejnym SaaS-em, który musisz subskrybować. Jest to **lokalne narzędzie desktopowe** - jak VS Code czy Terminal - które instalujesz raz i używasz bez ograniczeń. Płat tylko za API LLM (OpenAI/Anthropic) lub używaj darmowych lokalnych modeli (Ollama). Żadnych monthly fees, żadnych limitów, żadnego vendor lock-in.

Form factor - Desktop App

Aplikacja jest **natywną aplikacją desktopową** (nie webówka, nie PWA):

- **Okno główne:** trzy-panelowy layout (projekty | watki | chat)
- **Multi-window:** można otworzyć wiele okien (np. diff w osobnym oknie)

- **System tray:** działa w tle, szybki dostęp
- **Global shortcut:** `Cmd + Shift + C` (macOS) / `Ctrl + Shift + C` (Win/Linux) do otwarcia z dowolnego miejsca
- **Auto-updater:** aktualizacje bez reinstalu
- **Bundle size:** ~10-15 MB (vs 150MB+ Electron)

2. Design Aplikacji - Co Powinna Zawierac

Ta sekcja szczegółowo rozpisuje **kazdy element designu** aplikacji - od ogólnego layoutu po najmniejsze komponenty. To twój przewodnik po tym, co ma być zaimplementowane.

2.1. Architektura Okna i Layout

Główne Okno - Three-Panel Layout

Aplikacja opiera się na **trzech panelach** podobnie jak T3 Code:

- **Lewy panel (SIDEBAR)**: lista projektów + nawigacja główna (min 200px, max 350px, resizable)
- **Środkowy panel (THREADS)**: lista wątków/konwersacji dla wybranego projektu (min 250px, max 400px, resizable)
- **Prawy panel (MAIN)**: obszar roboczy - chat, diff viewer, terminal (reszta dostępnej przestrzeni)

Wszystkie panele są **resizable** (drag divider) i **collapsible** (można zwinąć każdy panel do ikon).

Top Bar (Header)

Pasek na samej górze okna (nad panelami):

- **Lewa strona**: nazwa aplikacji + ikona, aktualny projekt (breadcrumb)
- **Srodek**: nazwa aktywnego wątku, wskaźnik statusu agenta (idle/running/waiting)
- **Prawa strona**: selector modelu (dropdown), przycisk nowego wątku, przycisk ustawień (gear), przycisk minimalizacji/zamknięcia (native window controls)

Bottom Bar (Status Bar)

Pasek statusu na dole okna:

- **Lewa strona**: status połączenia (online/offline), wersja aplikacji
- **Srodek**: aktualnie używany model (np. "gpt-5.2-codex"), liczba aktywnych agentów
- **Prawa strona**: token usage (input/output), koszt sesji (USD), liczba plików w kontekście

Native Window Controls

- **macOS:** traffic lights (zamknij/minimalizuj/maksymalizuj) po lewej, przezroczyste tło, vibrancy effect
- **Windows:** standardowe przyciski po prawej, Mica/Acrylic material
- **Linux:** standardowe GTK decorations, dark theme default
- **Keyboard shortcuts:** `Cmd/Ctrl + ,` (ustawienia), `Cmd/Ctrl + N` (nowy watek), `Cmd/Ctrl + Shift + N` (nowy projekt), `Cmd/Ctrl + T` (nowy watek), `Cmd/Ctrl + W` (zamknij watek), `Cmd/Ctrl + B` (pokryj/ukryj sidebar)

2.2. Panel Projektu (Sidebar)

Sekcja Projekty

Lewy sidebar zawiera:

- **Przycisk "Add Project":** otwiera dialog (modal) do dodania projektu - można wybrać folder przez systemowy file picker lub wkleić ścieżkę absolutną
- **Lista projektów:** każdy projekt to "card" z:
 - Favicon/ikonka (z repozytorium lub generowana z pierwszej litery nazwy)
 - Nazwa projektu (bold)
 - Ścieżka (truncated, szary kolor)
 - Branch (np. "main", badge z kolorem)
 - Liczba aktywnych wątków (badge)
 - Status ostatniego wątku (checkmark, spinner, lub kropka)
- **Aktywny projekt** jest podświetlony (primary color background + border-left)
- **Kontekstowe menu** (prawy przycisk myszy): Open in Finder/Explorer, Remove from list, Refresh git status, Open in VS Code

Sekcja Nawigacji (ponizej projektów)

- **Skills:** ikona + napis, prowadzi do managera skilli
- **Settings:** ikona + napis, prowadzi do ustawień aplikacji
- **separator** (linia)

- **Help & About:** ikona + napis

2.3. Panel Watkow (Thread List)

Header Panelu Watkow

- **Nazwa projektu** (bold) jako tytul panelu
- **Przycisk "New Thread":** plus icon, tworzy nowy watek
- **Przycisk "Quick Actions":** play icon, lista zdefiniowanych akcji
- **Search bar:** filtrowanie watkow po nazwie/tresci (debounced)

Lista Watkow

Kazdy watek to "card" z:

- **Ikonka statusu:** spinner (running), checkmark (completed), clock (pending), chat bubble (chat mode)
- **Tytul watku:** auto-generowany z pierwszego promptu (np. "Add authentication middleware") lub manualnie edytowany
- **Podtytul:** data ostatniej aktywnosci (relative: "2 min ago") lub model uzyty
- **Badge trybu:** "Chat", "Plan", "Worktree" - kolorowe etykiety
- **Status PR:** jesli watek zakonczył sie PR, pokazuje status (open/merged/closed) z ikonka GitHub

Grupowanie watkow:

- "Running" (aktywne, na gorze)
- "Today"
- "Yesterday"
- "This Week"
- "Older"

Kontekstowe Menu Watku

- Rename
- Duplicate (kopiuj watek z ta sama konfiguracja)

- Archive
- Delete (z potwierdzeniem)
- Copy link (do worktree/diff)
- Open worktree in VS Code

2.4. Panel Chatu (Główny)

Konfiguracja Watku (nad chatem)

Gorna czesc panelu przed wiadomosciami - konfiguracja dla tego watku:

- **Model Selector:** dropdown z dostepnymi modelami (OpenAI GPT-5.2, Claude 4 Sonnet, Claude 4 Opus, Ollama local). Zapisane per watek.
- **Mode Toggle:** Chat mode | Plan mode (przycisk toggle/tab). Chat = szybkie pytania, Plan = agent najpierw robi plan przed pisaniem kodu.
- **Access Level:** Full Access | Supervised (toggle). Full = agent moze pisac pliki bez pytania. Supervised = kazda zmiana wymaga akceptacji.
- **Reasoning Level:** slider 1-5 (ile planowania agent robi przed dzialaniem)
- **Worktree toggle:** czy agent pracuje w izolowanym worktree

Obszar Wiadomosci

Scrollowalny obsrodek z lista wiadomosci:

- **User Message:** prawo, tlo primary/10, avatar usera (inicjaly), tresc jako markdown
- **Assistant Message:** lewo, tlo neutral, avatar AI (bot icon), tresc jako markdown z code blocks
- **Tool Call Message:** lekki styl, ikona wrench, nazwa toolu (np. "read_file: src/auth.ts"), status (running/completed/error), expandable szczegoly
- **Plan Message:** specjalny format dla Plan mode - lista krokow z checkboxami, kazdy krok moze byc expanded (pokazuje szczegoly)
- **System Message:** center, muted, italic (np. "Agent started worktree session on branch ai/feature-auth")

Code Blocks w Wiadomosciach

Kazdy blok kodu w markdown ma:

- **Header:** nazwa jezyka (badge) + nazwa pliku (jesli podana) + Copy button
- **Syntax Highlighting:** Monaco Editor w read-only mode (albo Prism.js jako fallback)
- **Line numbers:** opcjonalne (toggle w ustawieniach)
- **Diff highlighting:** jesli blok zawiera diff (lines z + i -), pokoloruj zielono/czerwono
- **"Apply" button:** jesli code block to zmiana do pliku, przycisk "Apply to file" (aplikuje zmiane do repozytorium)

Input Area

Na dole panelu chatu:

- **Textarea:** auto-resize (min 1 linia, max 10 linii), placeholder: "Ask the agent to do something..."
- **Attach button:** paperclip icon, pozwala dolaczyc pliki jako kontekst (drag & drop tez dziala)
- **Send button:** arrow icon, wysyla wiadomosc (disabled gdy empty)
- **Stop button:** square icon, pojawia sie gdy agent generuje odpowiedz (przerywa generation)
- **"@" mention:** wpisanie "@" otwiera autocomplete z skillami i plikami projektu
- **Keyboard:** **Enter** = send, **Shift** + **Enter** = nowa linia

2.5. Diff Viewer i Code Review

Diff Viewer

Komponent do przegladania zmian (jak GitHub diff):

- **Tryby widoku:** Split (side-by-side) | Unified (jedna kolumna, jak git diff) - toggle w headerze
- **Lista plikow:** sidebar po lewej z lista zmienionych plikow (nazwa, liczba + i - linii)
- **Code highlighting:** dodane linie na zielono (+), usuniete na czerwono (-), kontekst normalnie
- **Inline comments:** mozna dodac komentarz do konkretnej linii (dla AI - "zmien to na...")
- **"Stage chunk" button:** przy kazdym hunk diffa przycisk "Stage" (dodaje do commitu) lub "Revert" (cofa zmiane)
- **Navigation:** przyciski prev/next file, prev/next hunk

- **Full file view:** opcja "View full file" - pokazuje cały plik z podświetlonymi zmianami

Review Changes Panel

Gdy agent zakończy pracę, pojawia się panel review:

- **Podsumowanie:** ile plików zmienionych, ile + i - linii, lista zmian
- **Commit message:** auto-generowane przez AI (edytowalne)
- **Przyciski akcji:**
 - **Commit:** zapisuje zmiany w worktree
 - **Commit & Push:** commit + push do remote
 - **Commit, Push & Create PR:** jednym kliknięciem - commit, push, otwiera przeglądarkę z nowym PR (pre-filled title i description)
 - **Discard:** usuwa zmiany i worktree
 - **Continue:** agent kontynuuje pracę (dodatkowe zmiany)

2.6. Zintegrowany Terminal

Terminal Panel

Zintegrowany terminal na dole/do boku głównego panelu (jak VS Code):

- **xterm.js** jako renderer (WebGL acceleration)
- **Pty** (pseudo-terminal) zarządzany przez Rust backend
- **Shell:** używa domyślnego shella użytkownika (zsh, bash, fish, powershell)
- **Working directory:** auto-ustawiane na root projektu (lub worktree)
- **Tabs:** można mieć wiele terminali (plus icon w headerze)
- **Quick commands:** przyciski do częstych komend (np. "npm test", "npm run build", "git status")
- **AI integration:** można wybrać tekst w terminalu i wysłać do AI jako prompt ("explain this error", "fix this")

2.7. System Trybow Pracy

Chat Mode

Tryb dla szybkich pytan i iteracji:

- Agent odpowiada bez planowania (direct response)
- Nie tworzy planu, nie modyfikuje plikow automatycznie
- Moze czytac pliki (jesli pytanie o kod), ale nie pisze
- Idealny do: "co robi ta funkcja?", "jak zaimplementowac X?", "review tego kodu"
- Streaming odpowiedzi token-by-token

Plan Mode

Tryb dla zlozonych zadan wymagajacych planowania:

- Agent **najpierw analizuje codebase** i generuje plan krok-po-kroku
- Plan jest wyswietlany jako lista krokow z checkboxami
- Uzytkownik moze **edytowac plan** przed rozpoczeciem (dodawac/usuwac/zmieniac kroki)
- Po akceptacji planu, agent wykonuje go krok po kroku
- Kazdy krok moze byc zatwierdzony indywidualnie (w trybie Supervised)
- Idealny do: "dodaj system autentykacji", "zrefaktoryzuj ten modul", "dodaj testy do X"

Worktree Mode

Izolowane srodowisko pracy:

- Agent pracuje w **oddzielnym Git worktree** (osobny katalog, osobny branch)
- Twój glowny workspace **nie jest modyfikowany**
- Mozesz kontynuowac prace na main branch podczas gdy agent pracuje na swoim branchu
- Po zakonczeniu: diff + review + merge/PR
- Auto-cleanup: worktree jest usuwane po discard lub merge

Local Mode

Praca bezpośrednio w repozytorium:

- Agent modyfikuje pliki bezpośrednio w twoim workspace
- Szybszy (brak overheadu worktree)
- Ryzykowniejszy (brudzi workspace)
- Recommended tylko w trybie Supervised

2.8. System Skili i Quick Actions

Skills (Reusable Prompts)

System gotowych, reuzywalnych promptów:

- **Definicja skilla:** nazwa, opis, system prompt, dostępne tool'e, model
- **Wbudowane skille:**
 - `@react-component` - "Stwórz komponent React z TypeScript"
 - `@api-endpoint` - "Dodaj endpoint API z walidacją"
 - `@test-writer` - "Napisz testy jednostkowe dla tej funkcji"
 - `@refactor` - "Zrefaktoryzuj ten kod"
 - `@docs` - "Wygeneruj dokumentację dla tego modułu"
 - `@security` - "Przeskanuj kod pod kątem bezpieczeństwa"
 - `@optimize` - "Zoptymalizuj wydajność tego kodu"
- **Użycie:** wpisz `@nazwa-skilla` w inputcie chatu, auto-complete pokazuje dostępne skille
- **Custom skille:** można tworzyć własne skille w managerze (Settings > Skills)
- **Skill Manager:** lista skilli z opisem, edytor promptów, test playground

Quick Actions

Przyciski do natychmiastowego wykonania komendy shell:

- **Definicja:** nazwa, komenda shell, keybinding (opcjonalny), auto-run flag
- **Przykłady:** "Run Dev Server" (`npm run dev`), "Run Tests" (`npm test`), "Format" (`npm run format`), "Lint" (`npm run lint`)

- **Auto-run:** mozna ustawic, zeby komenda uruchamiala sie automatycznie po utworzeniu worktree (np. `npm install`)
- **UI:** przyciski w toolbarze watku, mozna dodac/edytowac/usuwac

2.9. Integracja Git i PR

Git Operations

- **Auto-detekcja:** kazdy projekt jest skanowany pod katem repozytorium Git
- **Branch indicator:** w sidebarze przy projekcie - aktualny branch
- **Git status:** ile zmienionych plikow, ile staged, ile untracked - badge
- **Commit:** auto-generowany message (przez AI) lub manualny, modal z lista zmian
- **Push:** push do remote z obsluga bledow
- **Create PR:** otwiera przegladarke z pre-filled PR (title, description z changes summary)

Worktree Management

- **Auto-create:** przy tworzeniu watku w worktree mode, auto tworzy branch `ai/[nazwa-watku]`
- **Auto-cleanup:** przy discard watku, usuwa worktree i branch
- **Worktree list:** mozna zobaczyc wszystkie worktree w ustawieniach projektu

2.10. Ustawienia i Konfiguracja

Settings Pages

- **General:** jezyk UI, theme (dark/light/system), font size, line height
- **AI Models:** lista skonfigurowanych modeli - dodawanie API keys (OpenAI, Anthropic), konfiguracja Ollama (URL, modele), default model per project
- **Skills:** manager skilli - lista, edytor, test playground, import/export
- **Projects:** lista dodanych projektow, default settings per project
- **Keyboard Shortcuts:** konfiguracja skrotow klawiszowych
- **Advanced:** max tokens, temperature default, timeouty, log level, data directory

3. Pełny Stack Technologiczny

Stack oparty na **Tauri v2** - frameworku do budowy desktop apps z React frontend i Rust backend. Zamiast Electron (120MB+), Tauri używa systemowego webview (WKWebView/WebView2/WebKitGTK) - aplikacja zajmuje **~10-15 MB**.

3.1. Desktop Shell - Tauri v2

Tabela 1 Tauri v2 - Komponenty

Komponent	Technologia	Rola
Desktop Shell	Tauri v2	Okna, IPC, system tray, menu, auto-updater, native APIs
Window Manager	tao (Tauri)	Tworzenie okien, multi-window, window state
WebView	WRY (Tauri)	Renderer: WKWebView (macOS), WebView2 (Win), WebKitGTK (Linux)
Auto Updater	tauri-plugin-updater	Aktualizacje OTA, cryptographic signatures
System Tray	tauri-plugin-tray	Ikona w trayu, menu kontekstowe
Global Shortcuts	tauri-plugin-global-shortcut	Cmd+Shift+C do otwarcia z dowolnego miejsca
Notifications	tauri-plugin-notification	Natywne powiadomienia OS (agent zakonczył prace)
Deep Links	tauri-plugin-deep-link	Protocol handler (codex://) dla linkow z przegladarki
Store	tauri-plugin-store	Key-value store dla ustawien aplikacji
Single Instance	tauri-plugin-single-instance	Tylko jedna instancja aplikacji
Dialog	tauri-plugin-dialog	Natywne okna dialogowe (file picker, confirm, message)
FS (sandboxed)	tauri-plugin-fs	Dostep do systemu plikow (scoped do wybranych folderow)
Shell	tauri-plugin-shell	Wykonywanie komend shell (git, npm, itp.)
HTTP	tauri-plugin-http	Requesty HTTP do API (OpenAI, Anthropic, Ollama)
Clipboard	tauri-plugin-clipboard-manager	Kopiowanie kodu, tekstu
OS	tauri-plugin-os	Info o systemie operacyjnym

3.2. Frontend - React 19

Tabela 2 Frontend Stack

Technologia	Wersja	Rola
React	19.x	UI framework - Server Components, Actions, use()
TypeScript	5.6+	Type safety end-to-end
Vite	6.x	Build tool (Tauri integruje sie natywnie z Vite)
Tailwind CSS	4.x	Utility-first CSS
shadcn/ui	latest	Komponenty UI (Dialog, Tabs, Resizable, Dropdown, itp.)
Zustand	5.x	State management (global store)
TanStack Query	5.x	Server state management (cache, refetch)
TanStack Virtual	3.x	Virtualizacja długich list (watki, wiadomosci)
Framer Motion	12.x	Animacje (paneli, przejsc, streaming tokens)
React Markdown	9.x	Renderowanie markdown w wiadomosciach
Prism.js / Shiki	latest	Syntax highlighting w markdown
Monaco Editor	0.50+	Code editor (VS Code editor w przegladance)
xterm.js	5.x	Terminal emulator (WebGL renderer)
React Diff Viewer	4.x	Diff viewer (split/unified)
Zod	3.24+	Walidacja schematow

3.3. Backend - Rust

Tabela 3 Rust Backend Stack

Technologia	Crate	Rola
Async Runtime	tokio	Async runtime (commands, I/O, background tasks)
Serialization	serde + serde_json	JSON serialization dla IPC (JS ↔ Rust)
Errors	thiserror + anyhow	Type-safe error handling
Git Operations	git2	Natywne operacje Git (libgit2 bindings) - branch, commit, worktree, diff, status
Process Management	tokio::process	Spawnowanie procesow (shell, npm, itp.)
Pty	portable-pty	Pseudo-terminal dla zintegrowanego terminala
SQLite	rusqlite + deadpool-sqlite	Embedded database (connection pooling)
HTTP Client	reqwest	HTTP client do API (OpenAI, Anthropic, Ollama)
LLM Streaming	eventsourcing-stream + futures	SSE streaming z API LLM
File Watching	notify	Monitoring zmian w plikach projektu (inotify/FSEvents)
Path Utils	dunce	Normalizacja sciezek cross-platform
Regex	regex	Parsowanie outputu komend
Logging	tracing + tracing-subscriber	Structured logging (do pliku + frontend)
Configuration	confy	Zarzadzanie configiem aplikacji

3.4. Baza Danych - SQLite

Tabela 4 SQLite Schema

Tabela	Pola Kluczowe	Cel
projects	id, name, path, git_branch, git_remote, created_at	Lista projektow/dodanych repozytoriow
threads	id, project_id, title, mode, model, status, worktree_path, branch_name, reasoning_level, created_at, updated_at	Watki konwersacji/agentow
messages	id, thread_id, role, content, metadata (JSON), tokens_input, tokens_output, created_at	Wiadomosci w watku
skills	id, name, description, system_prompt, tools, model, is_builtin, created_at	Reusable prompts
quick_actions	id, project_id, name, command, keybinding, auto_run, created_at	Szybkie komendy shell
settings	key, value	Ustawienia aplikacji (key-value)
api_keys	provider, key_encrypted, created_at	Zaszyfrowane klucze API (AES-256)
usage_stats	id, date, provider, model, tokens_input, tokens_output, cost_usd	Statystyki uzycia

3.5. AI / LLM Layer

Tabela 5 AI Layer

Komponent	Technologia	Rola
LLM Providers	OpenAI API, Anthropic API, Ollama API	Modele językowe - użytkownik podaje własne klucze
Streaming	SSE (Server-Sent Events)	Token-by-token streaming przez request + eventsource
Local LLM	Ollama (localhost:11434)	Darmowe lokalne modele (Llama, Mistral, CodeLlama, DeepSeek)
Context Builder	Custom Rust	Składanie kontekstu: system prompt + attached files + repo context + history
Token Counter	tiktoken-rs (OpenAI) / estimate (inny)	Liczenie tokenów przed wysłaniem (nie przekroczyć limitu context window)
Encryption	aes-gcm	Szyfrowanie kluczy API w SQLite (AES-256-GCM)

3.6. Code Editor - Monaco

Tabela 6 Code Editor Integration

Komponent	Opis
Monaco Editor	VS Code editor w przeglądarce (webpack/vite plugin)
Language Support	TypeScript, JavaScript, Python, Go, Rust, JSON, YAML, Markdown (wiele języków)
Themes	VS Code themes: Dark+ (default), Light+, Monokai, Dracula
Features	Syntax highlighting, error squiggles, hover info, go-to-definition (lokalne), minimap, line numbers
Diff Editor	Monaco diff editor dla porównywania plików (original vs modified)
Inline Edit	Edycja inline w diff viewer (można zmieniać kod przed approval)

3.7. Terminal - xterm.js

Tabela 7 Terminal Stack

Komponent	Technologia	Rola
Terminal Frontend	xterm.js 5.x	Renderer terminala (WebGL addon dla wydajności)
Fit Addon	xterm-addon-fit	Auto-dopasowanie do kontenera
WebLinks Addon	xterm-addon-web-links	Klikalne linki w terminalu
Search Addon	xterm-addon-search	Szukanie w terminalu
Pty Backend	portable-pty (Rust)	Pseudo-terminal zarządzany przez Rust
Shell	System shell	zsh/bash/fish (macOS/Linux), PowerShell/Git Bash (Windows)

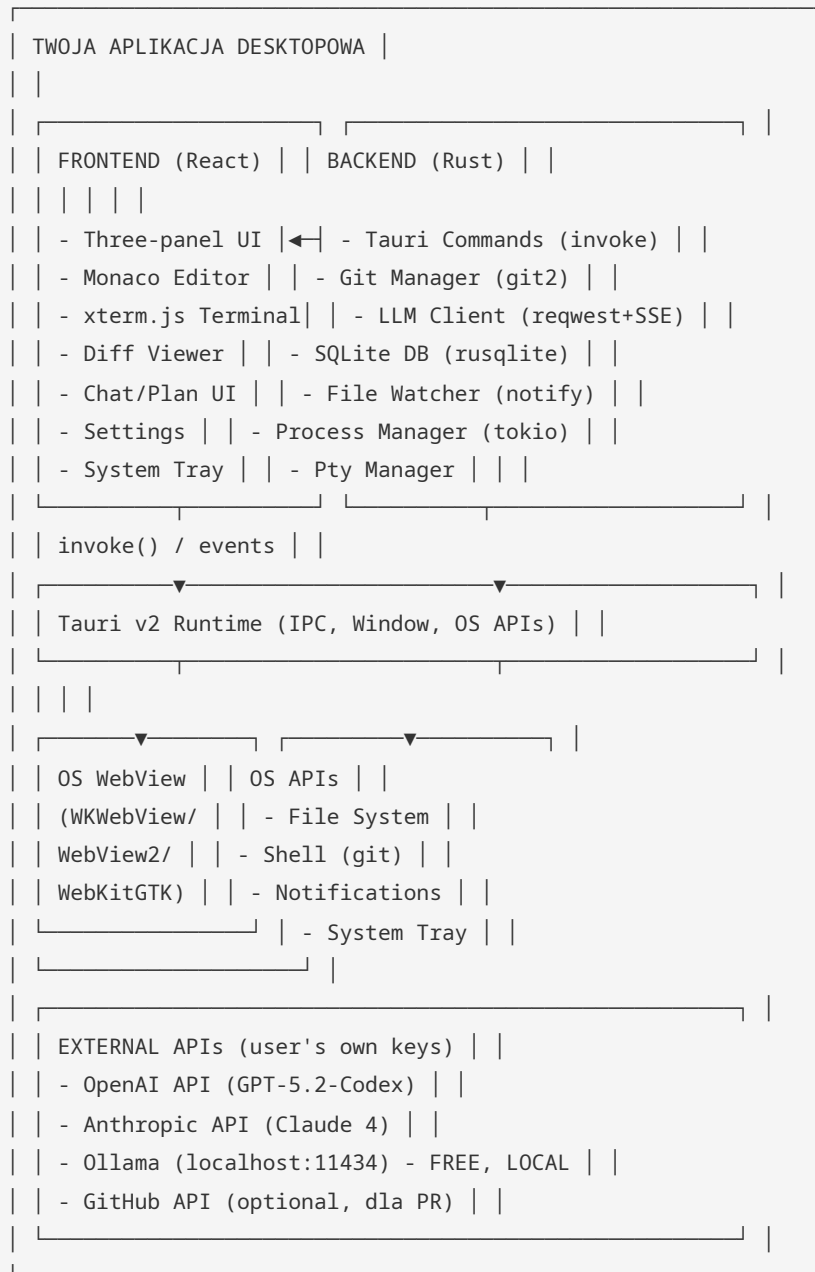
3.8. Infrastruktura i DevOps

Tabela 8 Infrastruktura

Komponent	Technologia	Rola
Build	Tauri CLI + Vite	<code>cargo tauri build</code> - buduje dla wszystkich platform
CI/CD	GitHub Actions	Automated builds: macOS (universal), Windows, Linux (ApplImage + deb)
Updater Server	GitHub Releases + JSON	Static JSON manifest dla Tauri auto-updater
Signing	Apple Developer ID, Windows EV cert	Podpisywanie binarek (wymagane dla auto-updatera)
Distribution	GitHub Releases	.dmg (macOS), .msi/.exe (Windows), .ApplImage/.deb (Linux)
Package Size	Tauri optimization	~10-15 MB (vs 150MB+ Electron)

4. Architektura Systemu

Diagram Architektury



Kluczowe Komponenty Rust (Backend)

Tabela 9 Rust Modules

Modul	Opis	Kluczowe Funkcje
<code>git_manager</code>	Operacje Git przez git2	init, status, branch, worktree_create, worktree_delete, commit, diff, log
<code>llm_client</code>	Klient API dla LLM	stream_chat, count_tokens, format_messages, handle_sse_stream
<code>db</code>	SQLite database layer	CRUD projects, threads, messages, skills, settings
<code>file_watcher</code>	Monitoring plików	watch_project, on_change callback, debounced events
<code>process_manager</code>	Zarządzanie procesami	spawn_shell, kill_process, read_output, write_input
<code>pty_manager</code>	Pseudo-terminal	create_pty, resize, write, read_stream
<code>crypto</code>	Szyfrowanie	encrypt_api_key, decrypt_api_key (AES-256-GCM)
<code>config</code>	Konfiguracja	load_settings, save_settings, defaults
<code>tray</code>	System tray	setup_tray, handle_events, show/hide window

Flow Danych (Przykład)

Użytkownik wysyła prompt w chatcie:

- Frontend (React): `sendMessage(content, threadId)` wywołuje `invoke("llm_chat", payload)`
- Tauri IPC: serializuje payload do JSON, przesyła do Rust
- Rust `llm_chat` command: (a) pobiera historie z SQLite, (b) buduje kontekst (system prompt + attached files), (c) liczy tokeny, (d) wysyła request HTTP do API (OpenAI/Anthropic/Ollama) z streaming (SSE)
- Rust streamuje odpowiedź: każdy token jest wysyłany jako event przez `app_handle.emit("llm_token", token)`
- Tauri IPC: event jest przesyłany do frontendu
- Frontend (React): odbiera event przez `listen("llm_token", handler)` i appenduje token do wiadomości
- Po zakończeniu: Rust zapisuje pełną odpowiedź do SQLite

5. Roadmap Projektowy

Roadmap podzielony na **6 faz** i **16 sprintow** (kazdy sprint = 2 tygodnie). Calkowity czas: **32 tygodnie (8 miesiecy)**. Dla solo developera pracujacego w wolnym czasie - realistyczna timeline.

Tabela 10 Podsumowanie Roadmap

Faza	Sprinty	Czas	Cel	MVP?
Faza 0: Setup	1	2 tyg.	Scaffold Tauri v2 + React, tooling	
Faza 1: Core UI	2-3	4 tyg.	Three-panel layout, resizable panels	
Faza 2: Chat AI	4-5	4 tyg.	Chat streaming, markdown, code blocks	MVP v1
Faza 3: Projekty + Git	6-7	4 tyg.	Project management, git integration	MVP v2
Faza 4: Agenci + Worktree	8-10	6 tyg.	Plan mode, git worktree, diff viewer	Beta
Faza 5: Terminal + Skille	11-13	6 tyg.	Terminal, skills, quick actions	
Faza 6: Hardening	14-16	6 tyg.	Auto-updater, polish, release	v1.0

6. Szczegolowy Opis Sprintow

Faza 0: Setup i Scaffold

Sprint 1 | Tydzie 1-2 | Cel: Projekt Tauri v2 + React gotowy do developmentu

Sprint 1: Project Scaffold

Tydzie 1-2 | Solo dev

TASK 1.1: Inicjalizacja Tauri v2 + React

Utworzenie projektu przez `npm create tauri-app@latest` z wybranymi opcjami: React, TypeScript, Vite. Struktura katalogow: `src/` (frontend React), `src-tauri/` (backend Rust). Weryfikacja: `cargo tauri dev` uruchamia aplikacje z hot-reload. Konfiguracja `tauri.conf.json`: window size (1400×900), title bar style (overlay dla macOS), allowlist capabilities (fs, shell, http, dialog, notification, clipboard, os, process).

TASK 1.2: Konfiguracja Frontend Tooling

Instalacja i konfiguracja: Tailwind CSS 4 z `@tailwindcss/vite`, shadcn/ui (init + pierwsze komponenty: Button, Input, Dialog, Tabs, ScrollArea, DropdownMenu, Tooltip, Resizable), Zustand (global store), Framer Motion. Setup aliasow importow w `vite.config.ts` (`@/` → `src/`) i `tsconfig.json`. Konfiguracja ESLint + Prettier z auto-formatowaniem na save.

TASK 1.3: Konfiguracja Rust Tooling

Dodanie dependencies do `Cargo.toml`: `tokio` (async runtime), `serde` + `serde_json` (serialization), `thiserror` (errors), `rusqlite` (SQLite), `request` (HTTP client), `git2` (Git), `notify` (file watcher). Struktura modulow w `src-tauri/src/`: `main.rs`, `lib.rs`, `db.rs`, `git.rs`, `llm.rs`, `fs_watcher.rs`. Setup `tracing` do logowania.

TASK 1.4: SQLite Schema i Migrations

Stworzenie schematu SQLite z tabelami: `projects`, `threads`, `messages`, `skills`, `quick_actions`, `settings`, `api_keys`, `usage_stats`. System migracji: `db/migrations/001_init.sql` - pliki SQL aplikowane sekwencyjnie przy starcie aplikacji. Rust `db.rs`: inicjalizacja polaczenia, wykonywanie migracji, podstawowe CRUD funkcje. Db path: `app_data_dir/db.sqlite` (Tauri `appDataDir`).

TASK 1.5: System Tray i Okno

Konfiguracja system tray: ikona aplikacji, menu kontekstowe ("Open", "Settings", "Quit"). Event handling: klikniecie w ikone pokazuje/chowa okno. Blokowanie domyslnego zamknienia (close → hide, quit → exit). Global shortcut: `Cmd/Ctrl + Shift + C` do pokazania okna z dowolnego miejsca. Single instance lock (tylko jedna kopia aplikacji).

Kryteria Akceptacji Sprintu 1:

- Aplikacja sie kompiluje i uruchamia (`cargo tauri dev`)
- System tray dziala (ikona, menu, show/hide)
- SQLite jest inicjalizowany z pelnym schematem
- Global shortcut otwiera okno
- Hot-reload dziala dla frontend i backend

Faza 1: Core UI Layout

Sprint 2-3 | Tydzie 3-6 | Cel: Three-panel layout, nawigacja, resizable panels

Sprint 2: Three-Panel Layout

Tydzie 3-4 | Solo dev

TASK 2.1: Resizable Panel Container

Implementacja trzech-panelowego layoutu z `shadcn/ui Resizable` (lub `react-resizable-panels`): lewy panel (Projects, min 200px, max 350px), srodkowy panel (Threads, min 250px, max 400px), prawy panel (Main Content, flex-1). Panele sa resizable (drag dividers) i collapsible (przyciski zwiń/rozwiń). Persist state: rozmiary paneli zapisywane w localStorage i odtwarzane przy starcie.

TASK 2.2: Project Sidebar (Lewy Panel)

Komponent `ProjectSidebar`: lista projektow z SQLite (mock dane na start). Kazdy projekt: favicon/icon (generowana z pierwszej litery), nazwa (bold), sciezka (truncated, gray), branch (badge). Przycisk "Add Project" - wywoluje Tauri `dialog.open` (folder picker) i zapisuje do SQLite. Aktywny projekt: podswietlenie primary color + border-left. Context menu (prawy klawisz): Remove, Refresh git status, Open in VS Code (przez `shell.open`). Pusta lista: "No projects yet" + CTA "Add your first project".

TASK 2.3: Thread List (Srodkowy Panel)

Komponent `ThreadList`: lista watekow dla wybranego projektu (z SQLite, mock dane). Kazdy watek: ikona statusu, tytul, podtytul (data/model), badge trybu (Chat/Plan). Grupowanie: "Running", "Today", "Yesterday", "This Week", "Older". Przycisk "New Thread" - tworzy nowy watek w SQLite i przelacza do niego. Search bar do filtrowania watekow. Pusta lista: placeholder z CTA.

TASK 2.4: Main Panel z Tabami

Komponent `MainPanel`: obszar roboczy z tabami (shadcn Tabs): "Chat", "Diff", "Terminal". Domyslnie "Chat" (pusty placeholder na razie). Header z nazwa watku, modelem (badge), i przyciskami (New Thread, Settings). Bottom bar (status): wersja app, polaczenie, token usage (placeholder).

TASK 2.5: Top Bar i Status Bar

`TopBar`: nazwa aplikacji + ikona, breadcrumb (Project / Thread), przycisk nowego watku, selector modelu (dropdown placeholder), settings gear. `StatusBar`: lewa strona (status online/offline), srodek (aktualny model, liczba aktywnych agentow), prawa strona (token usage placeholder). Sticky na dole okna.

Kryteria Akceptacji Sprintu 2:

- Trzy-panele sa widoczne i resizable (drag dividers)
- Mozna dodac projekt przez folder picker (zapisuje do SQLite)
- Lista projektow i watekow jest wyswietlana z bazy danych
- Mozna tworzyc nowe watki

- Panele mozna zwijac/rozwijac

Sprint 3: UI Polish i Theme

Tydzie 5-6 | Solo dev

TASK 3.1: Dark/Light Theme

Implementacja systemu themingu przez `next-themes` (lub custom): dark (default), light, system. Kolory dostosowane do obu motywow. Native window decorations: macOS (vibrancy, transparent titlebar), Windows (Mica material). Persist theme w Tauri Store. Toggle w settings.

TASK 3.2: Native Window Controls

macOS: traffic lights po lewej, titlebar hidden (overlay style), vibrancy effect. Windows: standardowe kontrolki, custom titlebar dla spójności. Linux: GTK decorations. Keyboard shortcuts: `Cmd/Ctrl + ,` (settings), `Cmd/Ctrl + N` (nowy watek), `Cmd/Ctrl + Shift + N` (nowy projekt), `Cmd/Ctrl + B` (toggle sidebar). Skroty konfigurowalne w settings.

TASK 3.3: Settings UI

Strona ustawien (modal lub osobny widok): General (theme, font size), AI Models (lista skonfigurowanych - placeholder), Skills (placeholder), Projects (lista, mozna usuwac), Keyboard Shortcuts (placeholder), Advanced (placeholder). Zapis ustawien do SQLite + Tauri Store.

TASK 3.4: Context Menu i Popovers

Kontekstowe menu (prawy klawisz) dla: projektow (Remove, Open in Finder, Open in VS Code), watkow (Rename, Duplicate, Archive, Delete), wiadomosci (Copy, Delete). shadcn DropdownMenu + shadcn ContextMenu.

TASK 3.5: Empty States i Loading

Spojne empty state components: "No projects yet" z CTA, "No threads" z CTA, "Select a project" (gdy brak wybranego projektu). Loading skeletons (shadcn Skeleton) dla list podczas fetchowania. Animacje przejsc (Framer Motion): fade-in dla nowych elementow, slide dla paneli.

Kryteria Akceptacji Sprintu 3:

- Dark/light theme dziala i jest persistowane
- Window controls wygladaja natywnie na kazdej platformie
- Settings UI jest funkcjonalne (theme, projects list)
- Context menu dziala na projektach i watkach
- Empty states i loading states sa spojne

Faza 2: Chat AI ze Streaming

Sprint 4-5 | Tydzie 7-10 | Cel: Pelny chat z markdown, code blocks, streaming, multi-provider

Sprint 4: Chat Backend (Rust)

Tydzie 7-8 | Solo dev

TASK 4.1: LLM Client w Rust

Modul `llm.rs` : struktura `LlmClient` z konfiguracja (provider, api_key, model, base_url). Funkcja `stream_chat(messages, config)` wysyla POST do API (OpenAI/Anthropic/Ollama) z `stream: true` i zwraca `impl Stream` (SSE). Uzycie `reqwest` + `eventsource-stream` + `futures`. Parsowanie SSE: kazdy event zawiera `choices[0].delta.content` (OpenAI) lub `delta.text` (Anthropic). Error handling: retry 3x, timeout 30s, graceful degradation.

TASK 4.2: Tauri Command dla Chatu

```
#[tauri::command] async fn llm_chat(window: Window, payload: ChatPayload) -> Result<(), String> :
```

(1) pobiera historie messages z SQLite, (2) buduje pelny payload (system prompt + history + user message), (3) liczy tokeny, (4) wysyla request streaming, (5) dla kazdego tokena: `window.emit("llm_token", json!({"thread_id": id, "token": token}))`, (6) po zakonczeniu zapisuje pelna odpowiedz do SQLite. Wsparcie dla Abort (przerwanie generowania).

TASK 4.3: API Key Management

Modul `crypto.rs` : szyfrowanie kluczy API. Szyfr AES-256-GCM z kluczem derivowanym z machine-specific secret (hardware ID). Tabela `api_keys` : provider (openai/anthropic/ollama), key_encrypted (BLOB), created_at. Commands: `save_api_key(provider, key)`, `get_api_key(provider)` (zwraca odszyfrowany), `delete_api_key(provider)`. Nigdy nie zapisuj plain-text keys!

TASK 4.4: Model Config UI

Frontend: Settings > AI Models - formularz do dodawania kluczy API. Pola: Provider (dropdown), API Key (password input, masked), Model (dropdown z lista), Base URL (optional, dla Ollama). Zapis przez Tauri command `save_api_key`. Lista dodanych modeli z opcja edycji/usuwania. Test connection button (ping API z kluczem).

TASK 4.5: Ollama Integration

Wsparcie dla Ollama (localhost:11434): auto-discovery (sprawdza czy Ollama dziala na porcie 11434), lista dostepnych modeli (`GET /api/tags`), konfiguracja bez klucza API (Ollama nie wymaga). Default dla offline mode. Fallback: gdy Ollama nie jest dostepne, pokazuje info + link do instalacji.

Kryteria Akceptacji Sprintu 4:

- Rust command `llm_chat` streamuje odpowiedz z OpenAI API
- Tokeny sa emitowane jako Tauri events do frontendu
- Klucze API sa szyfrowane w SQLite
- Ollama jest wykrywane i dziala jako provider

Sprint 5: Chat UI i Markdown

Tydzie 9-10 | Solo dev

TASK 5.1: Chat Input Component

Komponent `ChatInput` : textarea auto-resize (min 1 linia, max 10 linii), placeholder "Ask the agent to do something...". Przycisk Attach (paperclip icon) - Tauri `dialog.open` do wyboru pliku, plik jest odczytywany przez `fs.readFile` i dodawany jako kontekst (wyswietlany jako chip z nazwa + opcja usuniecia). Przycisk Send (arrow icon, disabled gdy pusty). Przycisk Stop (square icon, pojawia sie podczas streamingu). Keyboard: Enter = send, Shift+Enter = nowa linia.

TASK 5.2: Message List Component

Komponent `MessageList` : lista wiadomosci z virtualizacja (TanStack Virtual) dla wydajnosci przy 100+ wiadomosciach. Typy wiadomosci: User (prawo, primary tint), Assistant (lewo, neutral), System (center, muted italic), Tool Call (lekki, wrench icon). Auto-scroll do najnowszej wiadomosci z opcja wyłączenia (przycisk pojawia sie przy scroll up). Timestamp: relative ("2m ago").

TASK 5.3: Markdown Rendering

Integracja `react-markdown` : renderowanie naglowkow, list, linkow, tabele (shadcn Table), blockquotes. `remark-gfm` dla GitHub Flavored Markdown. Code blocks: `react-syntax-highlighter` (Prism.js) z theme dostosowanym do dark/light. Code block header: jezyk (badge) + Copy button (z checkmark po skopiowaniu). Inline code: bg tint. Linki: otwieraja sie w przegladarce (`shell.open`).

TASK 5.4: Streaming UI

Frontend odbiera tokeny przez `listen("llm_token", handler)` i appenduje do aktualnej wiadomosci assistant. Animacja: tokeny pojawiaja sie bez flickering (Framer Motion layout animation). Typing indicator: trzy kropki animowane gdy czekamy na pierwszy token. Stop button: wysyla abort signal (command `llm_abort`). Cursor blink na koncu streamingu. Po zakonczeniu: pelna wiadomosc jest zapisywana do SQLite (async).

TASK 5.5: Thread Config Bar

Gorna czesc panelu chatu - konfiguracja watku: Model Selector (dropdown z skonfigurowanymi modelami), Mode Toggle (Chat | Plan - tab switch), Access Level (Full | Supervised - toggle), Reasoning Level (slider 1-5, tylko dla Plan mode). Wszystkie wartosci zapisywane w SQLite per watek i odtwarzane przy otwarciu.

Kryteria Akceptacji Sprintu 5 (MVP v1!):

- Chat UI pozwala wyslac wiadomosc i dostac streaming odpowiedz
- Odpowiedzi sa renderowane jako markdown z code blocks
- Code blocks maja syntax highlighting i copy button

- Można załaczyc plik jako kontekst
- Stop button przerywa generowanie
- Wiadomosci sa persistowane w SQLite (pozniej odswiezone)

Milestone: MVP v1

Po tym sprincie masz dzialajaca aplikacje desktopowa z chat AI, streaming, markdown i multi-provider support. To jest minimalny produkt, ktorem mozesz sie chwalic i uzywac na co dzien!

Faza 3: Projekty i Git Integration

Sprint 6-7 | Tydzie 11-14 | Cel: Pelna integracja z Git, worktree, project management

Sprint 6: Git Integration

Tydzie 11-12 | Solo dev

TASK 6.1: Git Manager (Rust)

Modul `git.rs` uzywajacy `git2`: funkcje: `repo_status(path)` zwraca branch, ahead/behind, modified/staged/untracked files; `get_branches(path)` lista branchy; `create_worktree(repo_path, branch_name, worktree_path)` tworzy nowy worktree; `delete_worktree(repo_path, worktree_path)` usuwa worktree; `commit(path, message, files)` commituje zmiany; `push(path, branch)` push do remote; `get_diff(path, old_tree, new_tree)` zwraca diff jako string. Error handling: kazda funkcja zwraca `Result<T, GitError>`.

TASK 6.2: Project Git Status

Po dodaniu projektu, aplikacja auto-detektuje repozytorium Git (sprawdza czy `.git` istnieje). W sidebarze przy projekcie: aktualny branch (badge), liczba zmienionych plikow (badge). Tauri commands: `get_git_status(project_id)` - Rust uzywa `git2` do pobrania statusu i zwraca JSON. Auto-refresh: co 30s + na demand (prawy klawisz "Refresh").

TASK 6.3: Add Project Flow

Ulepszony flow dodawania projektu: (1) Przycisk "Add Project" otwiera Tauri `dialog.open` (folder picker), (2) Uzytkownik wybiera folder, (3) Rust sprawdza czy to Git repo (`git2`), (4) Pobiera branch i remote URL, (5) Zapisuje do SQLite, (6) Frontend odswieza liste. Toast notification: "Project added: [nazwa]". Blad: jesli folder nie istnieje lub nie ma praw dostepu.

TASK 6.4: File Attachment Context

Ulepszenie attach w chacie: mozna dolaczyc plik z projektu (relative path) lub z dowolnego miejsca (absolute path). Kontekst jest budowany w Rust: `build_context(attached_files, project_path)` - czyta pliki, formatuje jako ````filename\ncontent\n````. Token counting: sprawdza czy kontekst + prompt nie przekracza limitu modelu. Warning jesli za duzy kontekst.

TASK 6.5: Thread Mode Config

Implementacja config bar w watku: Model selector (pobiera z SQLite lista skonfigurowanych modeli), Mode toggle (Chat/Plan) zapisany w `threads.mode`, Access Level (Full/Supervised) w `threads.access_level`, Reasoning Level slider w `threads.reasoning_level`, Worktree toggle (enabled/disabled). Wartosci domyslne: Chat mode, Supervised access, Reasoning 3, Worktree enabled.

Kryteria Akceptacji Sprintu 6:

- Rust git manager dziala (status, worktree, commit, push)
- Projekty pokazuja branch i status Git w sidebarze
- File attachment z projektu buduje kontekst dla AI

- Thread config (mode, access, reasoning) jest persistowane

Sprint 7: Worktree i Auto-Actions

Tydzie 13-14 | Solo dev

TASK 7.1: Worktree Auto-Create

Gdy uzytkownik tworzy watek w worktree mode: Rust automatycznie tworzy worktree przez `git2::Worktree::add()`. Branch name: `ai/[slug-tytulu-watku]` (np. `ai/add-auth-middleware`). Worktree path: `[repo-path]/.ai-worktrees/[branch-name]`. Zapis `worktree_path` i `branch_name` w tabeli threads. Watek jest powiazany z tym worktree - wszystkie operacje agenta dzialaja w tej sciezce.

TASK 7.2: Worktree Cleanup

Auto-cleanup worktree: gdy watek jest archiwowany/usuniety lub uzytkownik klika "Discard" - Rust usuwa worktree przez `git2::Worktree::prune()` + usuwa branch. Opcja "Keep worktree" w dialogu potwierdzenia. Manual cleanup w Settings > Projects: lista worktrees z opcjami usuniecia.

TASK 7.3: Agent File Operations

Tauri commands dla operacji na plikach (uzywane przez agenta AI): `read_file(path)` - czyta plik jako string, `write_file(path, content)` - zapisuje plik, `list_files(dir)` - lista plikow w katalogu (tree view), `search_files(dir, query)` - szuka tekstu w plikach (grep-like). Scope: tylko wewnatrz `project_path/worktree_path` (security!). File watcher (notify): emituje event gdy agent zmienia plik (frontend odswieza diff).

TASK 7.4: Quick Actions UI

Przycisk "Quick Actions" w toolbarze watku: dropdown z lista akcji. Wbudowane akcje: "Run Dev Server" (`npm run dev`), "Run Tests" (`npm test`), "Run Lint" (`npm run lint`), "Run Format" (`npm run format`). Definiowanie wlasnych akcji: Settings > Quick Actions - formularz (nazwa, komenda, keybinding, auto-run). Akcje sa persistowane w SQLite. Output akcji: wyswietlany w terminal panelu.

TASK 7.5: Commit, Push, PR Flow

Po zakonczeniu pracy agenta: panel "Review Changes" z lista zmienionych plikow (z git diff stat). Przyciski: "Commit" (modal z auto-generated message lub manual), "Commit & Push", "Commit, Push & Create PR" (otwiera przegladarke z pre-filled PR title i description). Tauri `shell.open` do otwarcia GitHub w przegladarce. PR URL: `https://github.com/[owner]/[repo]/compare/[branch]?title=...&body=...`.

Kryteria Akceptacji Sprintu 7 (MVP v2!):

- Worktree jest auto-tworezone dla watkow w worktree mode
- Agent moze czytac i pisac pliki w worktree
- Quick actions dzialaja i wykonuja komendy shell
- Commit + Push + Create PR flow dziala

Faza 4: Agenci, Plan Mode i Diff Viewer

Sprint 8-10 | Tydzie 15-20 | Cel: Plan mode, diff viewer, code review, agent actions

Sprint 8: Plan Mode i Diff Viewer

Tydzie 15-16 | Solo dev

TASK 8.1: Plan Mode Backend

Gdy watek jest w Plan mode, agent przed dzialaniem generuje plan: system prompt modyfikowany o "First, analyze the codebase and create a step-by-step plan. Do not write any code yet.". Streaming planu: kazdy krok jest emitowany jako osobny event. Format planu: JSON array krokow (title, description, files_involved). Zapis planu w `threads.plan` (JSON). Frontend wyswietla plan jako checkliste.

TASK 8.2: Plan UI

Komponent `PlanView` : lista krokow z checkboxami (pending/in_progress/completed). Kazdy krok: tytul (bold), opis (zwijany), lista plikow (badges). Przyciski: "Edit Plan" (edytowalny JSON), "Approve & Execute" (agent zaczyna wykonywac kroki), "Regenerate" (generuje plan od nowa). Progress bar: X of Y steps completed. Podczas wykonywania: aktywny krok ma spinner, zakonczone maja checkmark.

TASK 8.3: Diff Viewer Component

Komponent `DiffViewer` oparty na `react-diff-viewer-continued` (lub Monaco diff editor). Tryby: Split (side-by-side) | Unified (jedna kolumna). Lista plikow po lewej z liczba + i - linii. Code highlighting: dodane linie (green bg), usuniete (red bg), kontekst (normal). Header: nazwa pliku, liczba zmian. Navigation: prev/next file, prev/next hunk.

TASK 8.4: Agent Actions (Tool Calls)

System tool calls: agent moze wywolywac funkcje (przez function calling w API): `read_file`, `write_file`, `run_command`, `search_code`. Kazdy tool call jest wyswietlany w chacie jako specjalna wiadomosc (tool call card): nazwa toolu, parametry (JSON), status (running/completed/error), wynik (zwijany). Timeout: 30s per tool call. Error handling: retry lub przerwanie.

TASK 8.5: Human-in-the-Loop (Supervised Mode)

Gdy watek jest w Supervised mode: przed kazda operacja piszaca (`write_file`, `run_command` destrukcyjne) agent zatrzymuje sie i czeka na akceptacje. Modal "Action Required": opis akcji, podglad zmian (diff), przyciski "Approve" (kontynuuj), "Reject" (pomin), "Edit" (zmien i approve). Timeout: 10 minut (po czym anuluje). Wszystkie oczekujace akcje sa persistowane (mozna zamknac i wrocic do aplikacji).

Kryteria Akceptacji Sprintu 8:

- Plan mode generuje i wyswietla plan krok-po-kroku
- Diff viewer pokazuje zmiany w split i unified mode
- Tool calls sa wyswietlane w chacie ze statusem
- Supervised mode blokuje akcje i czeka na approval

Sprint 9: Code Review i Inline Edit

Tydzie 17-18 | Solo dev

TASK 9.1: Inline Comments w Diff

Mozliwosc dodawania komentarzy do konkretnych linii w diff viewer (jak GitHub PR review). Klikniecie na numer linii otwiera textarea. Komentarze sa powiazane z linia i plikiem. Komentarze moga byc wyslane do AI jako kontekst ("fix this line"). Wyswietlanie komentarzy jako badge/bubble przy linii.

TASK 9.2: Inline Edit w Diff

Edycja kodu bezposrednio w diff viewer (przed approval). Podwojne klikniecie na linie (lub przycisk "Edit") otwiera Monaco editor inline. Mozna zmienic kod i zapisac - zmiana jest uwzgledniona w finalnym diff. Przydatne gdy agent prawie dobrze napisal, ale trzeba cos poprawic.

TASK 9.3: Review Changes Panel

Panel review po zakonczeniu pracy agenta: podsumowanie (N plikow, +X/-Y linii), lista zmian (kazdy plik z mini-diff), auto-generated commit message (editable), auto-generated PR description. Przyciski: "Stage All", "Unstage All", "Stage Selected", "Commit", "Commit & Push", "Create PR", "Discard All". Status: open/merged/closed indicator.

TASK 9.4: Git Diff per Turn

Diff viewer pokazuje zmiany per "turn" (krok agenta) - mozna zobaczyc co agent zmienil w kazdym kroku osobno. Timeline: lista krokow z mini-diff stat. Klikniecie kroku pokazuje diff dla tego konkretnego kroku. Pomaga zrozumiec jak agent doszedl do finalnego rozwiazania.

TASK 9.5: File Watcher i Auto-Refresh

File watcher (notify crate) monitoruje project path. Gdy agent zmienia plik: (1) emit Tauri event `file_changed`, (2) frontend odswieza diff, (3) status bar pokazuje "X files changed". Debounced (500ms) dla batch changes. Mozna wylaczyc w settings.

Kryteria Akceptacji Sprintu 9:

- Inline comments i edit w diff viewer dzialaja
- Review changes panel pokazuje podsumowanie i akcje
- Diff per turn pokazuje zmiany per krok agenta
- File watcher auto-odswieza diff

Sprint 10: Multi-Window i Polish

Tydzie 19-20 | Solo dev

TASK 10.1: Multi-Window Support

Tauri multi-window: mozna otworzyc diff w osobnym oknie ("Open in New Window" w kontekstowym menu). Nowe okno: tytul "Diff: [filename]", zawiera tylko diff viewer (bez sidebarow). Komunikacja miedzy oknami przez Tauri events. Zamkniecie okna nie zamyka glownej aplikacji. Window state (pozycja, rozmiar) persistowane.

TASK 10.2: Global Shortcut Enhancements

Cmd/Ctrl + Shift + C: pokazuje/chowa okno (toggle). **Cmd/Ctrl + Shift + N**: szybkie tworzenie nowego watku (nawet gdy app jest schowana). Global shortcut konfigurowalny w Settings. Tray click: toggle window. Tray menu: Recent Projects, New Thread, Settings, Quit.

TASK 10.3: Notifications

Natywne powiadomienia OS (tauri-plugin-notification): gdy agent zakonczy prace ("Task completed: [tytul]"), gdy oczekuje na approval ("Action required"), gdy wystapi blad. Powiadomienia klikalne (otwieraja odpowiedni watek). Mozna wylaczyc w settings per typ.

TASK 10.4: Performance Optimization

Virtualizacja list: TanStack Virtual dla watkow (100+ watkow) i wiadomosci (1000+ wiadomosci). Lazy loading: diff viewer laduje tylko widoczne pliki. Code splitting: Monaco editor ladowany on-demand (dynamic import). Debounced resize handler dla resizable panels. SQLite queries z indeksami (threads.project_id, messages.thread_id).

TASK 10.5: Beta Testing

Przygotowanie builda beta: `cargo tauri build` dla wszystkich platform. GitHub Release z opisem i changelog. Zebranie feedbacku (wewnetrzne testowanie). Lista znanych bugow i ograniczen. Zbieranie metryk: crash reports, performance data.

Kryteria Akceptacji Sprintu 10 (BETA!):

- Multi-window dziala (diff w osobnym oknie)
- Powiadomienia dzialaja (agent completed, action required)
- Aplikacja jest plynnie przy duzej liczbie watkow/wiadomosci
- Beta build gotowy do testowania

Faza 5: Terminal, Skille i Quick Actions

Sprint 11-13 | Tydzie 21-26 | Cel: Zintegrowany terminal, system skilli, quick actions

Sprint 11: Integrated Terminal

Tydzie 21-22 | Solo dev

TASK 11.1: Pty Backend (Rust)

Modul `pty.rs` uzywajacy `portable-pty`: `create_pty(shell, cwd, cols, rows)` tworzy pseudo-terminal. `write(input)` wysyla input do pty. `read_stream()` zwraca stream outputu (async). `resize(cols, rows)` zmienia rozmiar. Kill: zabija proces. Tauri commands: `pty_create`, `pty_write`, `pty_resize`, `pty_kill`. Events: `pty_output` (data z pty do frontendu), `pty_exit` (proces sie zakonczy).

TASK 11.2: Terminal Frontend

Komponent `TerminalPanel` oparty na `xterm.js`: `xterm` + `WebGL` addon (wydajny renderer) + `Fit` addon (auto-resize) + `WebLinks` addon (klikalne linki). Polaczenie z backendem: `WebSocket`-like przez Tauri events (zamiast prawdziwego WS - uzywamy Tauri IPC z streaming). Input z `xterm.js` wysylany przez `invoke("pty_write")`. Output z `listen("pty_output")` renderowany w `xterm`. Tabs: mozna miec wiele terminali (plus icon w headerze), kazdy z osobnym pty.

TASK 11.3: Terminal Integration

Working directory: auto-ustawiane na root projektu (lub worktree). Shell: auto-detekcja domyslnego shella (`SHELL` env var, lub fallback: `bash/zsh/powershell`). Quick commands: przyciski w toolbarze terminala do czestych komend ("`npm test`", "`npm run build`", "`git status`"). AI integration: zaznaczenie tekstu w terminalu + opcja "Explain" / "Fix" wysyla do AI.

TASK 11.4: Terminal UI

Panel terminala: tabs na gorze (przelaczanie miedzy terminalami), plus icon (nowy terminal), close icon (zamyka). Przycisk "Clear" (czysci ekran), "Kill" (zabija proces). Terminal zajmuje dolna czesc main panelu (jak VS Code) - mozna go pokazac/ukryc przyciskiem lub skrotem `Ctrl/Cmd + ``. Resizable height (drag).

Kryteria Akceptacji Sprintu 11:

- Terminal dziala (shell, komendy, output)
- Wiele terminali w tabs
- Working directory auto-ustawiane na projekt
- Quick commands dzialaja

Sprint 12: Skills System

Tydzie 23-24 | Solo dev

TASK 12.1: Skills Data Model

Tabela `skills` : id, name, description, system_prompt (text), tools (JSON array), model (string), is_builtin (boolean), created_at. Built-in skills (seed): @react-component, @api-endpoint, @test-writer, @refactor, @docs, @security, @optimize. Custom skills: tworzone przez uzytkownika.

TASK 12.2: Skill Manager UI

Strona Settings > Skills: lista skilli (karty z nazwa, opisem, tag built-in/custom). Edytor: formularz (nazwa, opis, system prompt textarea, tools multi-select, model select). Test playground: textarea do wpisania promptu testowego, przycisk "Run", wynik w panelu chat. Import/export: JSON format.

TASK 12.3: @ Mention w Chat

W inpucie chatu: wpisanie "@" otwiera autocomplete popup z lista skilli (fuzzy search). Wybór skilla dodaje go do kontekstu (chip z nazwa). Skill modyfikuje system prompt dla tego zapytania. Mozna uzyc wielu skilli naraz. Usuniecie chipusu = usuniecie skilla z kontekstu.

TASK 12.4: Skill Execution

Rust: gdy wiadomosc zawiera skill, `build_context` dodaje system prompt skilla do payloadu LLM. Format: "You are an expert [skill description]. [system_prompt]. The user request: [user message]". Priority: jesli wiele skilli, skille sa scalane (concatenated system prompts).

TASK 12.5: Skills Polish

7 wbudowanych skilli z dobrze napisanymi promptami. Testy kazdego skilla w playground. Dokumentacja skilli w UI (opis, przyklady uzycia). Presets: przyciski szybkiego uzycia skilli nad inputem (jak emoji reactions, ale skille).

Kryteria Akceptacji Sprintu 12:

- 7 wbudowanych skilli dziala
- Mozna tworzyc custom skille
- @ mention autocomplete w chacie
- Skille modyfikuja zachowanie AI

Sprint 13: Quick Actions i File Explorer

Tydzie 25-26 | Solo dev

TASK 13.1: Quick Actions Backend

Tabela `quick_actions` : `id`, `project_id`, `name`, `command`, `keybinding`, `auto_run`, `created_at`.

Commands: `create_quick_action` , `list_quick_actions(project_id)` , `run_quick_action(id)` (spawna komende shell w terminalu), `delete_quick_action(id)` .

Auto-run: po utworzeniu worktree, akcje z `auto_run=true` sa automatycznie wykonywane (np. `npm install`).

TASK 13.2: Quick Actions UI

Toolbar w headerze watku: lista quick actions jako przyciski (ikona + nazwa) + dropdown "More". Przycisk "Add Action" otwiera formularz (nazwa, komenda, keybinding checkbox, auto-run checkbox). Keybinding: jesli ustawione, dziala globalnie w kontekscie projektu. Output komendy: wyswietlany w terminal panelu.

TASK 13.3: File Explorer

Komponent `FileExplorer` w sidebarze (opcjonalny panel): tree view plikow projektu (jak VS Code explorer). Tauri `fs.readDir` rekurencyjnie buduje drzewo. Ikony per typ pliku. Klikniecie: otwiera plik w code viewer (Monaco). Context menu: Copy path, Open in VS Code. Auto-refresh: file watcher odswieza drzewo przy zmianach. Collapse/expand all.

TASK 13.4: Code Viewer

Komponent `CodeViewer` : Monaco Editor w read-only mode do przegladania plikow. Otwiera sie po kliknieciu pliku w explorer lub w chacie ("show file"). Syntax highlighting dla wielu jezykow. Go to line (Ctrl+G). Breadcrumbs: sciezka pliku. Split view: mozna otworzyc wiele plikow w tabs (jak VS Code).

TASK 13.5: Usage Stats

Tabela `usage_stats` : `data`, `provider`, `model`, `tokens_input`, `tokens_output`, `cost_usd`. Dashboard w Settings: wykres uzycia tokenow w czasie, koszty per model, total cost. Lokalne liczenie (nie wysyla danych nigdzie). Export do CSV.

Kryteria Akceptacji Sprintu 13:

- Quick actions sa wykonywane w terminalu
- File explorer pokazuje tree view projektu
- Code viewer (Monaco) otwiera pliki do przegladania
- Usage stats sa zbierane i wyswietlane

Faza 6: Hardening i Release

Sprint 14-16 | Tydzie 27-32 | Cel: Auto-updater, CI/CD, signing, produkcyjny release

Sprint 14: Auto-Updater i CI/CD

Tydzie 27-28 | Solo dev

TASK 14.1: Auto-Updater

Konfiguracja tauri-plugin-updater w `tauri.conf.json`: update server URL (GitHub Releases JSON manifest). Format manifestu: `{"version": "1.0.1", "notes": "Changelog...", "pub_date": "2026-05-01", "platforms": {"darwin-aarch64": {"signature": "...", "url": "https://..."}, "darwin-x86_64": {...}, "windows-x86_64": {...}, "linux-x86_64": {...}}}`. Signing: klucz prywatny (ed25519) do podpisywania update'ow. Check: przy starcie app sprawdza czy jest nowsza wersja, pokazuje dialog z changelog + "Update Now" / "Later". Background download i install.

TASK 14.2: GitHub Actions CI/CD

Workflow `release.yml`: trigger na push tagu `v*`. Jobs: (1) `build-macos` - buduje universal binary (arm64 + x86_64), .dmg, (2) `build-windows` - buduje .msi + .exe, (3) `build-linux` - buduje .AppImage + .deb. Kazdy job: checkout, setup Rust + Node, `cargo tauri build`, upload artifact, tworzy GitHub Release z assetami. Code signing: macOS (Developer ID certificate), Windows (signtool z certyfikatem).

TASK 14.3: Error Reporting

Integracja z Sentry dla desktop (voluntary opt-in): `@sentry/electron` (lub `@sentry/browser` dla Tauri). Crash reporting: uncaught exceptions w Rust i JS. Error boundaries w React. Logi: tracing w Rust zapisuje do `app_logsDir/app.log` (rotacja: 5 plikow po 10MB). Możliwość eksportu logow (Settings > Advanced > Export Logs).

TASK 14.4: Data Backup

Export danych: Settings > Advanced > Export Data - generuje ZIP z: SQLite database, settings.json, api_keys (zaszyfrowane). Import danych: przy pierwszym uruchomieniu na nowej maszynie - można zaimportować ZIP. Auto-backup: co tydzień, snapshot DB do `app_dataDir/backups/` (max 10 snapshots, FIFO).

TASK 14.5: Documentation

README.md: opis, screenshoty, instalacja, stack, contributing. CHANGELOG.md: format Keep a Changelog. User Guide: docs/usage.md (getting started, features, tips). Architecture: docs/architecture.md (opis modułow, flow danych). License: MIT (open source).

Kryteria Akceptacji Sprintu 14:

- Auto-updater sprawdza i instaluje update'y
- CI/CD buduje dla wszystkich 3 platform
- Error reporting działa (voluntary)
- Export/import danych działa

Sprint 15: Polish i Bug Fixes

Tydzie 29-30 | Solo dev

TASK 15.1: Cross-Platform Testing

Testowanie na wszystkich platformach: macOS (Intel + Apple Silicon), Windows 10/11, Linux (Ubuntu 22.04). Sprawdzenie: native window controls, system tray, global shortcuts, file picker, shell commands, fonts rendering, dark/light theme. Znalezione bugi: lista + fixy.

TASK 15.2: Performance Audit

Startup time: target < 2s od klikniecia do interaktywnego UI. Memory usage: target < 200MB RAM (vs 500MB+ Electron). Bundle size: target < 15MB. SQLite queries: EXPLAIN dla wolnych zapytan, dodanie indeksow. Frontend: Lighthouse audit, eliminacja re-renderow (React DevTools Profiler).

TASK 15.3: Edge Cases

Handling edge cases: brak internetu (offline mode z Ollama), pusty projekt (brak plikow), bardzo duzy projekt (10000+ plikow - lazy loading), bardzo dluga konwersacja (1000+ wiadomosci - virtualizacja), nieprawidlowy klucz API (graceful error), Ollama nie zainstalowane (link do instrukcji), brak Git w projekcie (info + CTA).

TASK 15.4: Accessibility

A11y: keyboard navigation (Tab, Enter, Escape, Arrow keys), ARIA labels dla screen readerow, kontrast kolorow (WCAG AA), focus indicators, skroty klawiszowe dokumentowane. Test z VoiceOver (macOS) i NVDA (Windows).

TASK 15.5: Final Polish

Mikro-interakcje: Framer Motion dla wszystkich animacji (paneli, modalow, list). Loading states: skeleton screens, spinnery. Error states: friendly error messages z sugestiami fixow. Empty states: spójne i pomocne. Onboarding: pierwsze uruchomienie - wizard (witaj, dodaj projekt, skonfiguruj model).

Kryteria Akceptacji Sprintu 15:

- Aplikacja dziala na macOS, Windows i Linux
- Startup < 2s, RAM < 200MB
- Edge cases sa obslugiwane
- Onboarding dziala dla nowych uzytkownikow

Sprint 16: v1.0 Release!

Tydzie 31-32 | Solo dev

TASK 16.1: Release v1.0.0

Git tag `v1.0.0`. GitHub Release z: binarkami dla wszystkich platform, changelog, install instructions, screenshoty, feature list. Website/landing: prosta strona GitHub Pages z opisem aplikacji, download links, dokumentacja. Social: post na Twitter/X, Hacker News, Reddit r/webdev i r/rust. Show HN: "I built a free, open-source desktop AI code assistant".

TASK 16.2: Post-Release Monitoring

GitHub Issues: szablon dla bug reports i feature requests. Labels: bug, enhancement, question, good first issue. Milestones: v1.1, v1.2. Feedback collection: GitHub Discussions, ankieta (Google Forms). Metryki: liczba downloadow, gwiazdek, issue'ow.

TASK 16.3: v1.1 Planning

Backlog dla v1.1: wiecej modeli (Gemini, local via LM Studio), MCP (Model Context Protocol) integration, plugin system, team/organization features, cloud sync (opcjonalny). Priorytetyzacja na podstawie feedbacku uzytkownikow.

Kryteria Akceptacji Sprintu 16 (v1.0!):

- v1.0.0 jest wydane na GitHub z binarkami
- Strona landingowa dziala
- Feedback channel jest gotowy
- Backlog v1.1 jest zaplanowany

RELEASE v1.0!

Gratulacje! Twoja aplikacja desktopowa AI Code Assistant jest gotowa! 8 miesiecy pracy, 16 sprintow, i masz w pelni funkcjonalne narzedzie do pracy z AI kodujacym - takie jak T3 Code czy Codex App, ale w 100% twoje, open-source, i dzialajace lokalnie na twojej maszynie.

7. Wymagania Systemowe

7.1. Dla Developmentu

Tabela 11 Wymagania Dev

Komponent	Minimalne	Zalecane
OS	macOS 12+ / Windows 10+ / Linux Ubuntu 20.04+	macOS 14+ / Windows 11 / Ubuntu 22.04+
Procesor	4 rdzenie (Intel i5 / Apple M1)	8 rdzeni (Apple M3 / Intel i7 / AMD Ryzen 7)
RAM	8 GB	16 GB (32 GB dla Ollama 70B)
Dysk	10 GB SSD	50 GB SSD (NVMe)
Node.js	18.x	20.x LTS
Rust	1.75+	1.80+ (latest stable)
Git	2.30+	2.40+

7.2. Dla Uzytkownika (Runtime)

Tabela 12 Wymagania Runtime

Komponent	Minimalne	Zalecane
OS	macOS 11+ / Windows 10+ / Ubuntu 20.04+	macOS 14+ / Windows 11 / Ubuntu 22.04+
Procesor	2 rdzenie	4+ rdzeni
RAM	4 GB	8 GB (16 GB dla lokalnych LLM)
Dysk	50 MB (app)	500 MB (app + dane)
Internet	Wymagany dla cloud API	Wymagany dla cloud / lokalny dla Ollama

7.3. Szacowane Koszty (dla Ciebie jako Solo Deva)

Tabela 13 Koszty

Komponent	Koszt	Uwagi
Apple Developer ID	\$99/rok	Tylko jesli chcesz signing na macOS
Windows EV Certificate	\$200-400/rok	Opcjonalne (bez tego Windows Defender moze ostrzegac)
LLM API (OpenAI)	\$5-20/mies.	Zaleznie od uzycia (BYOK - platysz tylko za to co uzyjesz)
LLM API (Anthropic)	\$5-20/mies.	Opcjonalnie
Ollama (Local LLM)	\$0	Darmowe! Wymaga GPU dla wiekszych modeli
GitHub (hosting)	\$0	Public repo = free
Domain (opcjonalnie)	\$10-15/rok	Jesli chcesz strone landingowa
RAZEM (min)	\$0	Ollama + public repo = calkowicie za darmo
RAZEM (z API)	\$10-40/mies.	Z cloud API + macOS signing

Podsumowanie - Dlaczego Tauri v2?

Bundle: 10-15 MB (vs 150MB+ Electron) | **RAM: ~50-100 MB** (vs 200-500MB Electron) | **Startup: <1s** | **Native OS webview** (nie bundluje Chromium) | **Rust backend** (bezpieczenstwo, wydajnosc) | **Cross-platform** (macOS/Windows/Linux z jednego kodu) | **Open source i free**